

A classe Console do C#

Mário Leite

Quando surgiu o Windows® os desenvolvedores de sistema ficaram excitados”: uma nova maneira de criar programas em cores, com figuras, além de poderem manipular objetos reais nas interfaces. Entretanto, ainda hoje, após mais de 40 anos da novidade inicial, muitos programadores desconhecem um outro lado do de linguagens que implementam códigos em Windows/.NET: a classe **Console**.

Um caso muito interessante é o que acontece na excelente linguagem C#: sua classe **Console**, embora não ofereça telas coloridas, é uma ótima alternativa para implementar solução de “baixo custo” para o programador; com recursos muito potentes, e sem a necessidade de “perfumarias” desnecessárias. Esta postagem trata desse recurso de maneira bem didática, e em detalhes.

O projeto “**PrSaudaMundo**” com a aplicação para exibir as frases “Alô Mundo!” na tela, tem características básicas de qualquer projeto simples em console. Aqui foi usado método **Show()** da classe **MessageBox** definida no *namespace* **System.Windows.Forms**, adicionado ao projeto. O esquema da **figura 1** mostra os principais elementos que compõem um projeto *Console* básico.

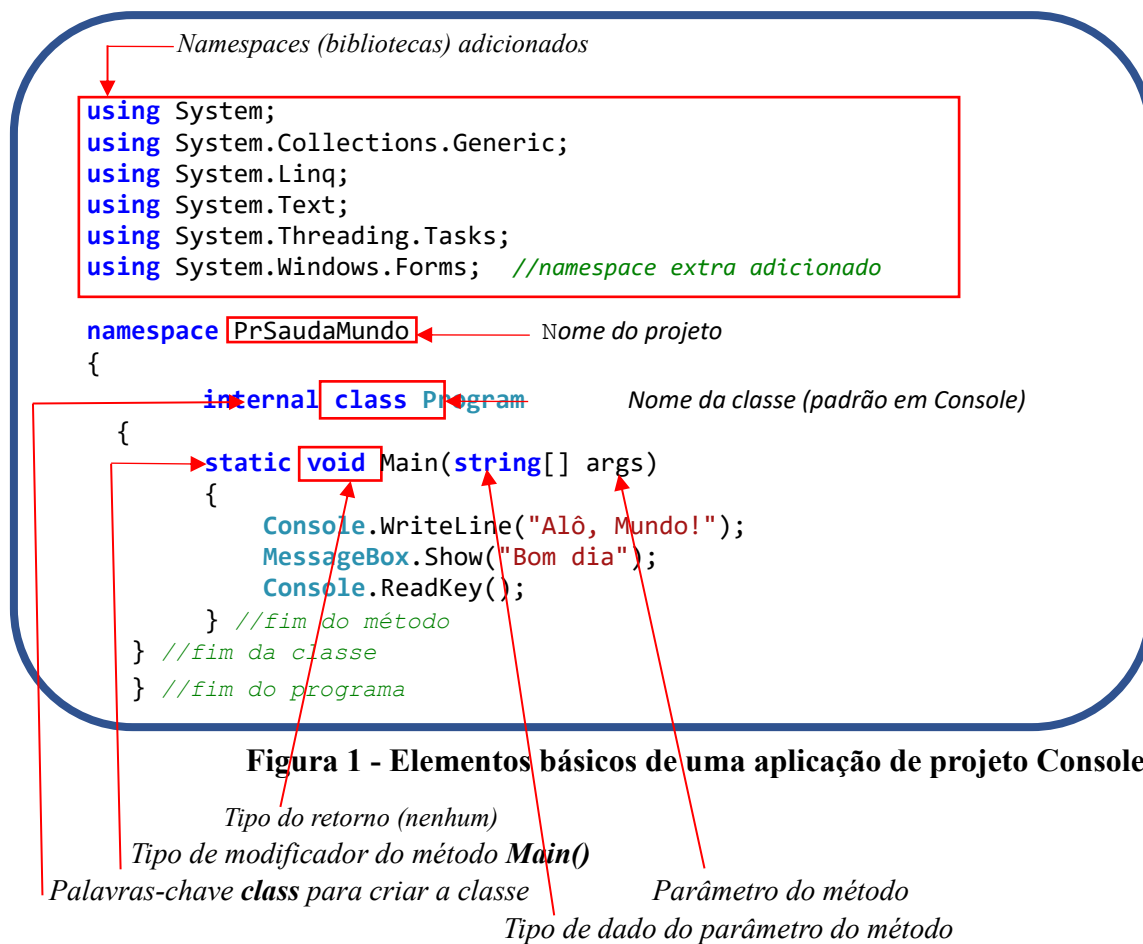


Figura 1 - Elementos básicos de uma aplicação de projeto Console

No contexto da OOP (**Object-Oriented Programming** - Programação Orientada a Objetos) a **figura 1** destaca as duas linhas de códigos com instruções que começam com a palavra-chave **Console**:

Diagram illustrating the components of C# code snippets:

- `Console.WriteLine("Alô, Mundo!");`
 - `Console`: classe
 - `.WriteLine`: método
 - `("Alô, Mundo!")`: parâmetro
- `Console.ReadKey();`
 - `Console`: classe
 - `.ReadKey`: método

De um modo geral, em C# uma classe é criada com a palavra-chave **class**. Seja, por exemplo, criar a classe **TCaixa** com as características (*atributos*) de um cubo. Para isto, o código abaixo pode ser utilizado:

```
internal class TCaixa
{
    private double lado;
    public double FunVolume()
    {
        return (lado*lado*lado);
    }
}
```

Diagram illustrating the components of the `TCaixa` class definition:

- `internal`: tipo de dado
- `class`: acessibilidades (modificadores)
- `TCaixa`: nome da classe
- `private`: acessibilidades (modificadores)
- `double`: tipo de dado
- `lado;`: atributo
- `public`: acessibilidades (modificadores)
- `double`: tipo de dado
- `FunVolume()`: método (sem parâmetros)
- `{`: início do bloco de código
- `return`: retorno (resultado do método)
- `(lado*lado*lado);`: expressão de cálculo
- `}`: fim do bloco de código

A criação de classes em C# é bem simples; entretanto, para usá-la efetivamente é necessário criar objetos (*instâncias*); e para isto é usado o operador **new**. Por exemplo, criar a instância **objCxPreta** da classe **TCaixa**.

```
TCaixa objCxPreta = new TCaixa();
```

Diagram illustrating the components of the object creation code:

- `TCaixa`: classe
- `objCxPreta`: instância
- `=`: operador de atribuição
- `new`: operador de criação de objeto
- `TCaixa()`: construtor da classe

A classe **Console**, que funciona como se fosse um objeto, possui *propriedades* e *métodos* responsáveis pelas características e ações pertinentes à uma figura geométrica. Para trabalhar com a classe **Console** deve ser usado o *namespace* **System** (que funciona como uma classe ancestral) presente nos projetos *Console* C#. Esta classe possui os métodos *Read()/ReadLine()* para *inputs* e *Write()/WriteLine()* para *outputs*. A rotina principal de uma aplicação de projetos deste tipo está contida no método **Main()** que é o “módulo” principal da aplicação.

Ao executar a aplicação o primeiro módulo (*método*) carregado é **Main()**; todos os outros são carregados, primeiramente, a partir dele. Observe o projeto “**PrExemplo.**” cujo protótipo de código é mostrado a seguir...

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace PrExemplo
{
    internal class Program

        static void Main(string[] args)
        {
            FunMenu(); //chama o método FunMenu()
            Console.ReadKey();
        }
        //-----
        public static void FunMenu()
        {
            //Aqui este método pode chamar qualquer um dos seguintes
            ...
            ...
        }
        //-----
        public int FunCalcMDC(int x, int y);
        {
            int mdc;
            ...
            ...
            return mdc;
        }
        //-----
        public double FunCalcRaiz(double n, double i);
        {
            double ind, raiz;
            ...
            ...
            return raiz;
        }
        //-----
        public static void FunInverteFrase(string f);
        {
            ...
            ...
        }
        //-----
        ---
        ---
    } //fim da classe
} //fim do programa (fim do namespace)

```

No esquema do projeto “**PrExemplo**” a aplicação contém cinco módulos:

- Módulo principal: **Main()**
- Módulo adicionado: **FunMenuPrinc()**
- Módulo adicionado: **FunCalcMDC()**
- Módulo adicionado: **FunCalcRaiz()**
- Módulo adicionado: **FunInverteFrase()**

Ao rodar a aplicação o método **Main()** chama o método **FunMenu()** que, por sua vez, pode chamar qualquer um dos outros métodos e esse método chamado pode chamar outros métodos, e assim por diante. No final o fluxo do programa sempre volta para o método **Main()** onde a aplicação se encerra. A última instrução do módulo principal deve ser **Console.ReadKey()** ou **Console.ReadLine()** para evitar o fechamento imediato da tela de resposta, e assim o usuário pode observar o resultado da aplicação, na **figura 2**, e só fechar após o pressionamento de alguma tecla

Nota: A classe **MessageBox** permite exibir mensagens com o seu método **Show()** em várias linhas; para isto, basta colocar **\n** para cada linha extra. Então, para exibir cinco linhas de mensagens teremos quatro **\n** concatenados com a mensagem normal. No exemplo abaixo temos três linhas de mensagens (uma extra) usando dois **\n**.

```
MessageBox.Show("E aí galera, beleza!?\n" + "Como vocês estão!?\n" + "Prontos para outra!?");
```

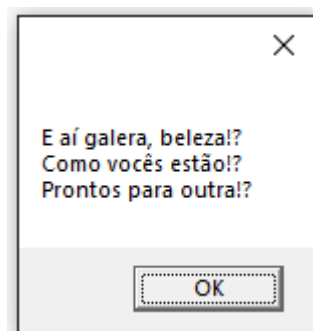


Figura 2 - Resultado